

Proyecto Final: Sistema de Consolidación de Ventas Omnicanal Super-Market

Geraldine Cornejo, Javiera Barrales, Rodrigo Flores

Junio 2026

1. Introducción al Proyecto

La corporación multinacional Super-Market enfrenta un desafío crítico de fragmentación de datos: las operaciones de Santiago, Lima y Buenos Aires generan información local en archivos CSV que presentan inconsistencias de formato, anomalías financieras y disparidad en las monedas.

Para proveer un repositorio de confianza al equipo de Data Science, implementamos una arquitectura Medallion. Esta solución centraliza y denormaliza las transacciones mediante Apache Beam (Capa Silver en Parquet), para luego modelar y estandarizar métricas financieras a USD mediante dbt en PostgreSQL (Capa Gold), bajo la orquestación resiliente de Apache Airflow.

2. Blueprint de Arquitectura y Flujo de Datos

A continuación, se presenta el Diagrama de Flujo de Datos (DFD) que ilustra el ciclo de vida de la información de forma segmentada.

3. Análisis Técnico y Decisiones Arquitectónicas

3.1. Contratos de Esquema y Calidad

Para proteger la integridad del dato desde su origen débil (CSV) hasta la capa analítica, definimos contratos estrictos. En Apache Beam, la estructura denormalizada se serializa utilizando el objeto `SILVER_SCHEMA` de PyArrow. Al pasarlo a la función `WriteToParquet`, PyArrow actúa como un guardián (gatekeeper) que rechaza cualquier escritura que no cumpla con los tipos y jerarquías anidadas.

En la Capa Gold, la calidad se garantiza matemáticamente mediante `schema.yml` en dbt. Aplicamos pruebas genéricas (`unique` y `not_null` sobre `transaction_id`) y usamos el paquete `dbt_expectations` para forzar que el `amount_usd` tenga un límite rígido de `min_value: 0.01`, asegurando que ninguna tasa fraccionaria anómala afecte los KPIs del negocio.

3.2. Orquestación del DAG

El grafo de Airflow se configuró secuencialmente: `Beam` $\rightarrow$ `Sensor` $\rightarrow$ `dbt`. Apoyándonos en la regla `all_success`, garantizamos que dbt se detenga por completo si los pasos anteriores fallan. Para aumentar la resiliencia, configuramos la tarea con 2 reintentos (`retries`) y un `retry_delay` de 5 minutos. Además, implementamos un `FileSensor` que hace **poking** cada 30 segundos verificando la creación exitosa del Parquet en la Capa Silver. Si transcurren 600 segundos (10 minutos), el sensor lanza un timeout y aborta el pipeline, evitando que dbt procese datos corruptos o incompletos.

3.3. Matriz de Resiliencia y Procesamiento Paralelo

Para la ingesta inicial, aprovechamos las capacidades distribuidas nativas de Apache Beam (específicamente mediante `beam.io.ReadFromText`), lo que nos permite un procesamiento óptimo y en paralelo de los chunks de texto sin la necesidad de implementar bloqueos manuales. Esto garantiza una lectura eficiente y libre de contención.

Sobre esta lectura paralela, aplicamos las reglas de validación: las ventas exigen 5 columnas y un `amount` estricto > 0 (casteado a float); los logs exigen 3 columnas y un dominio cerrado (`CREATED`, `PENDING`, `COMPLETED`, `REFUNDED`). Si una regla falla, el registro emite un `beam.pvalue.TaggedOutput('rejected', ...)`. Posteriormente, unificamos estos flujos de error con `beam.Flatten()` y los enviamos al archivo `rejected_sales.csv` para auditoría, manteniendo intacto el pipeline principal.

3.4. Sinking Strategy e Idempotencia

El uso de Apache Parquet permite compresión masiva y aislamiento por columnas. Para asegurar la idempotencia del pipeline en Beam, configuramos la escritura `WriteToParquet` sin el flag de **append**, forzando una sobreescritura limpia en cada ejecución diaria y estampando la metadata `processed_at`. En paralelo, la idempotencia en dbt (Capa Gold) se garantizó definiendo explícitamente `materialized='table'` en `fct_sales.sql`, lo que instruye a PostgreSQL a eliminar y recrear la tabla completa en cada corrida, eliminando el riesgo de duplicidad por **upserts** fallidos.

3.5. El Modelo de las 4 Preguntas de Beam

- **What (Qué se calcula):** Join de ventas e histórico de estados, denormalizándolos mediante tipado PyArrow estricto, aislando los datos anómalos.
- **Where (Dónde en el tiempo):** Uso de Ventanas Globales para el procesamiento Batch sobre datos limitados.

- **When (Cuándo se procesa):** Al finalizar el cierre contable diario.
- **How (Cómo se relacionan):** Usamos `CoGroupByKey` para agrupar las streams por `transaction_id`. Luego, la función `BuildNestedRecordFn` itera sobre estos agrupamientos, parseando las fechas string a objetos `datetime` UTC y anexándolos como diccionarios dentro de la lista `status_history`.

3.6. Repositorio

Nuestra solución se encuentra en: [Ir a GitHub](#)